Studio Nx

P1-23 화면 마무리 — 함수 설명 합본 (JS 96~101)

2026-10-13

목차

1. ai-notes/javascript/P1-23 - 화면 마무리/96_error_initErrorActions.md
2. ai-notes/javascript/P1-23 - 화면 마무리/97_home_loadDevNotes.md
3. ai-notes/javascript/P1-23 - 화면 마무리/98_home_initDevNotesModal.md
4. ai-notes/javascript/P1-23 - 화면 마무리/99_markdown_markdownToFragment.md
5. ai-notes/javascript/P1-23 - 화면 마무리/100_home_intro_sidenav_menu.md
6. ai-notes/javascript/P1-23 - 화면 마무리/101_privacy_initTocActiveState.md

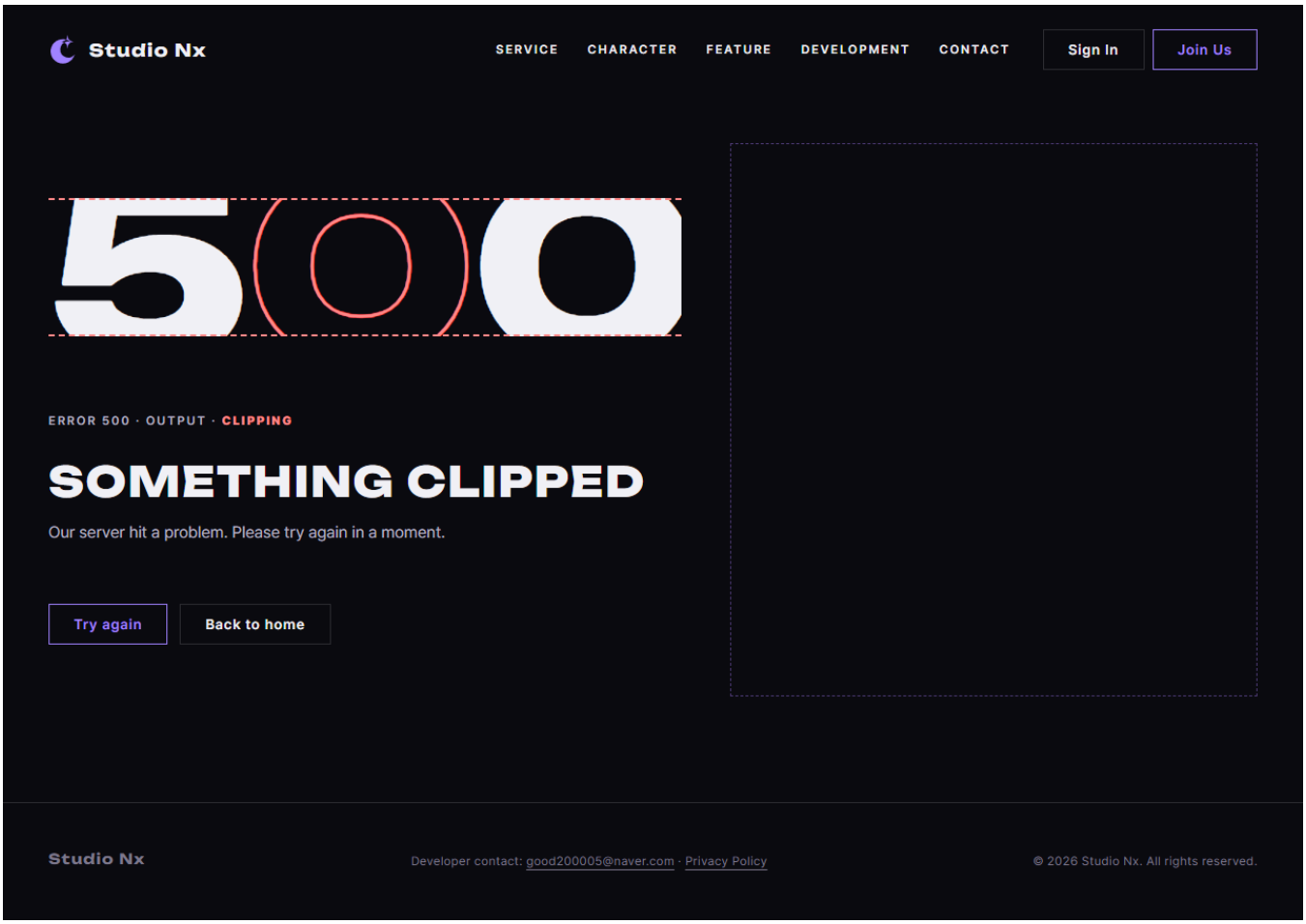
JS 96 — `initErrorActions()` (오류 페이지의 Go back, Try again 버튼)

항목	내용
파일	`studionx/frontend/static/js/pages/error.js`
작업	P1-23 화면 마무리: 오류 페이지의 두 버튼이 눌러도 아무 일도 하지 않던 것을 동작하게 함 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	(P1-23 첫 함수) · 97 `loadDevNotes()` (다음)

1. 하는 일

`errors/400 ~ 504.html` 일곱 페이지가 같이 쓰는 스크립트에서, 버튼 두 종류를 동작하게 합니다.

버튼	있는 페이지	동작
`Go back` (`data-action="back"`)	400, 403, 404	돌아갈 기록이 있으면 이전 페이지로, 없으면 `home.html`로
`Try again` (`data-action="retry"`)	429, 500, 503, 504	원래 하려던 일을 다시 시도 (아래 두 경우로 나뉩니다)
`Back to home` (일반 링크)	전부	링크 그대로 (JS가 건드리지 않음)



`Try again`이 두 경우로 나뉘는 이유: 오류 페이지가 화면에 나타나는 길이 두 가지라서, 똑같이 `reload`하면 한쪽에서 쓸모가 없습니다.

오류 페이지가 열린 방식	주소	`Try again`이 하는 일
서버가 그 주소에 **직접** 보냄 (P1-22)	원래 주소 그대로 (예: `/boom`)	`location.reload()`: 같은 주소를 다시 요청하므로 원래 요청이 다시 시도됩니다
화면의 JS가 **옮겨 옴** (`core/page-errors.js`, 예: 마이페이지의 API 실패)	`/errors/500.html`	`reload`하면 오류 페이지만 다시 뜨고 원래 일은 다시 안 합니다. 그래서 **어디서 왔는지(`document.referrer`)**를 보고 그 페이지로 돌아가 다시 시도합니다
위에서 온 곳을 알 수 없음 (주소를 직접 입력해서 옴 등)	`/errors/…`	`home.html`

2. 언제 쓰나

호출하는 곳	시점
페이지 맨 아래 `initErrorActions();`	오류 페이지가 열릴 때 한 번

3. 코드

```
export function initErrorActions() {
  document.addEventListener('click', function (event) {
    const button = event.target.closest('[data-action]');
    if (!button) return;
    const action = button.getAttribute('data-action');

    if (action === 'back') {
      if (history.length > 1) history.back();
      else location.href = 'home.html';
    } else if (action === 'retry') {
      if (location.pathname.indexOf('/errors/') !== 0) {
        location.reload();
      } else if (document.referrer && document.referrer.indexOf(location.origin + '/') === 0
        && document.referrer.indexOf(location.origin + '/errors/') !== 0) {
        location.href = document.referrer;
      } else {
        location.href = 'home.html';
      }
    }
  });
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export const initErrorActions = () => {
  document.addEventListener('click', ({ target }) => {
    const action = target.closest('[data-action]')?.dataset.action;
    if (action === 'back') {
      history.length > 1 ? history.back() : (location.href = 'home.html');
    } else if (action === 'retry') {
      const { origin, pathname } = location;
      const from = document.referrer;
      if (!pathname.startsWith('/errors/')) location.reload();
      else location.href = from.startsWith(`${origin}/`) && !from.startsWith(`${origin}/errors/`) ? from
: 'home.html';
```

```
    }
  });
};
```

최신 문법	실제 코드	뜻
`target.closest(...)?.dataset.action`	`button`이 없으면 `return`	없을 수 있는 값에서 안전하게 꺼냅니다 (옵셔널 체이닝)
`startsWith(...)`	`indexOf(...) === 0`	글자가 그것으로 시작하는지 봅니다
`` `${origin}/` ``	`location.origin + '/'`	글자에 값을 끼워 넣습니다 (템플릿 문자열)
`({ target }) =>`	`function (event)`와 `event.target`	화살표 함수와 구조 분해입니다

문법 풀이

식	뜻
`document.addEventListener('click', …)`	문서 어디든 클릭하면 부릅니다. 버튼마다 걸지 않고 **한 곳**에서 받습니다
`event.target.closest('[data-action]')`	누른 곳에서 위로 올라가며 `data-action` 속성이 있는 가장 가까운 요소를 찾습니다. 버튼 안의 글자를 눌러도 버튼을 찾습니다. 없으면 `null`입니다
`history.length`	이 탭의 기록 개수입니다. 1이면 돌아갈 곳이 없습니다
`location.pathname.indexOf('/errors/') !== 0`	주소의 경로가 `/errors/`로 **시작하지 않는지**입니다 (`indexOf`가 0이면 시작)
`document.referrer`	이 페이지로 오기 직전에 있던 페이지의 주소입니다. 모르면 빈 글자입니다
`location.origin`	사이트의 주소 부분입니다 (예: `http://localhost:8001`)

4. 흐름

1. 오류 페이지가 열리면 문서에 클릭 처리기를 하나 겁니다.
2. 클릭한 곳에서 `data-action`을 찾아, 없으면(일반 링크, 글자) 아무것도 하지 않습니다.
3. `back`이면 기록이 있을 때 뒤로, 없으면 홈으로 갑니다.
4. `retry`이면 주소가 `/errors/`로 시작하지 않을 때(서버가 직접 보낸 경우) 다시 불러오고, 시작하면 온 곳으로 돌아갑니다.

5. 왜 이렇게

선택	이유
버튼마다 걸지 않고 문서에 한 번	7개 페이지가 같은 스크립트를 쓰고, 버튼 위치가 페이지마다 다릅니다. 한 곳에서 받으면 어느 페이지에서나 동작합니다
`onclick`이 아니라 `addEventListener`	이 파일은 페이지마다 한 번 실행되어 겹쳐 걸릴 일이 없습니다
`Try again`을 두 경우로 나눔	`reload`만 쓰면 마이페이지에서 서버 오류가 난 뒤 `Try again`을 눌러도 오류 페이지만 다시 뜹니다 (실제 화면 시험 전에는 이 문제를 예상하지 못했고, 시험 설계 중에 발견했습니다)

선택	이유
`core/page-errors.js`(P1-17)를 고치지 않고 `document.referrer`를 씀	이동할 때 `location.replace`를 쓰므로 기록에는 오류 페이지만 남지만, 브라우저가 어디서 왔는지는 기억합니다. 이미 병합된 파일을 건드리지 않고 풀 수 있습니다
오류 페이지끼리 오가는 경우 제외 (`/errors/`로 시작하는 referrer)	오류 페이지에서 온 것을 "원래 페이지"로 보고 돌아가면 같은 곳을 맴돕니다
같은 사이트의 referrer만 믿음	다른 사이트에서 왔다면(링크 등) 그쪽으로 보내지 않고 홈으로 보냅니다
429, 503의 `Retry-After`로 버튼 잠시 막기는 하지 않음	선택 사항이었고, 서버가 지금 그 헤더를 보내지 않습니다

6. 시험 (실제 서버, 헤드리스 Chrome, 마우스로 버튼 클릭, 10개)

서버가 `/api/v1/me/dashboard`에 500을 주도록, 그리고 항상 오류를 던지는 `/boom` 주소를 임시로 붙인 실행 파일을 썼습니다 (시험 뒤 지움).

#	한 일	결과
E01	로그인한 상태로 마이페이지를 열었더니 서버 오류	`/errors/500.html`로 옮겨 옴 ✓
E02	`Try again`을 누름(서버는 여전히 오류)	마이페이지로 돌아갔다가 다시 오류 페이지 ✓
E03	서버를 회복시킨 뒤 `Try again`	마이페이지가 **정상으로** 열림 (프로필 이름 확인) ✓
E04	`/boom`(서버가 직접 보낸 500)을 엶	주소가 `/boom` 그대로 ✓
E05	`Try again`	같은 주소를 서버에 다시 요청함(요청 횟수가 정확히 1 증가) ✓
E06	주소창으로 `/errors/500.html`을 직접 열고 `Try again`	`home.html` ✓
E07	404 페이지의 `Go back` 버튼	있음 ✓
E08	홈 → 없는 주소 → `Go back`	`home.html`로 돌아감 ✓
E09	기록이 없는 상태(`history.length`가 1)에서 `Go back`	`home.html` ✓
E10	`Back to home` 링크	그대로 `home.html` ✓

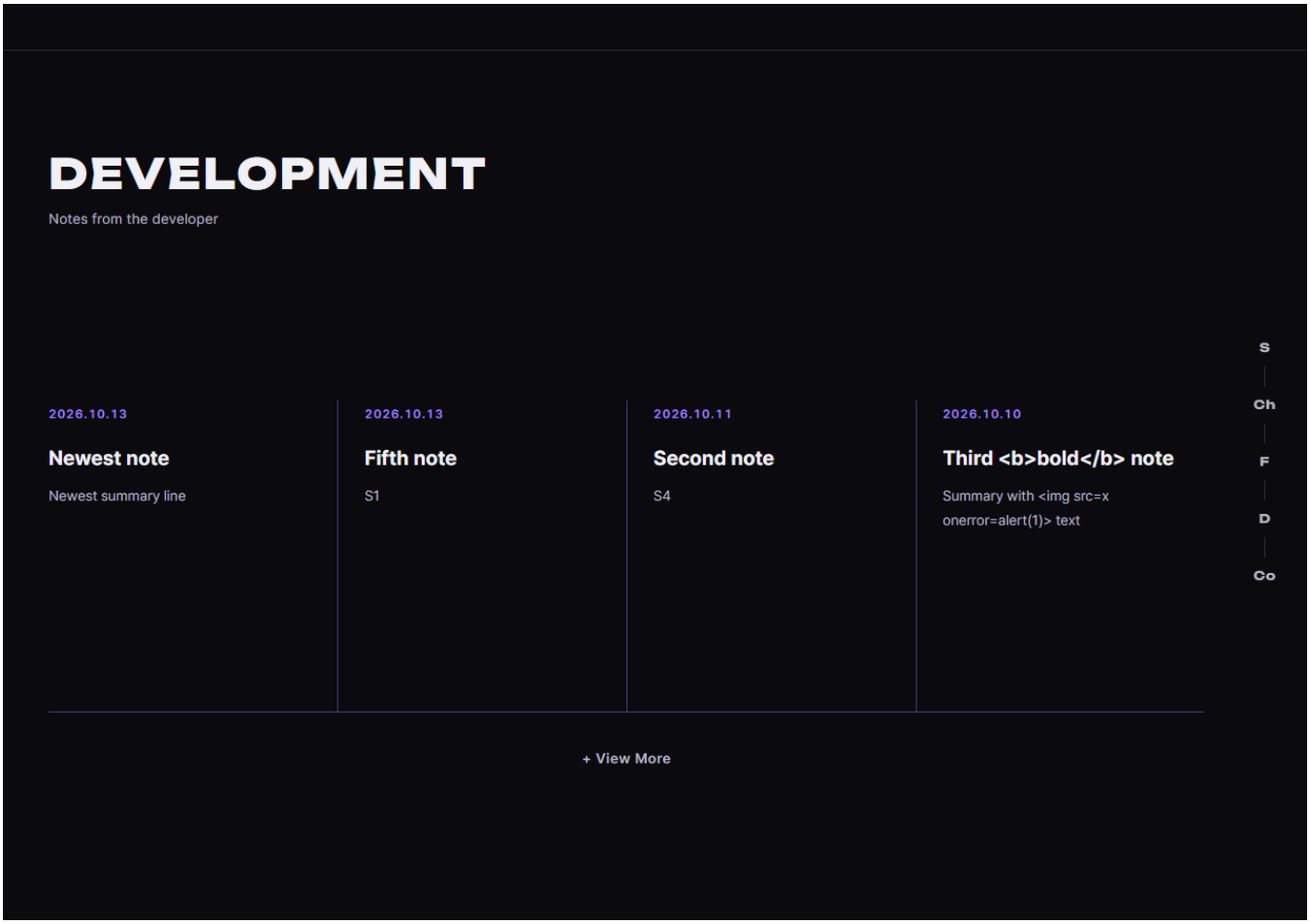
10개 모두 통과했습니다.

JS 97 — `loadDevNotes()`, `formatNoteDate()` (홈의 Development 공지 카드)

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`, `home.html`, `static/css/pages/home.css`
작업	P1-23 화면 마무리: 홈의 Development 카드 4개를 HTML의 예시 글자가 아니라 관리자가 쓴 실제 공지로 채우기 (PRD 01 FR-114)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	96 `initErrorActions()` · 98 View More 모달 (다음)

1. 하는 일

`GET /api/v1/dev-notes?limit=4`(P1-21의 85번)를 불러서 `.dev-notes` 목록에 카드 4개를 만듭니다. 카드 한 칸은 날짜, 제목, 요약입니다.



상황	화면
공지가 있음	최신 순으로 카드 최대 4개와 `+ View More`
공지가 하나도 없음	"No notes yet. Check back soon." (`+ View More`는 숨김)
불러오지 못함 (서버 오류, 네트워크 끊김)	"Notes are unavailable right now. Please try again later." (`+ View More`는 숨김)

상황	화면
응답 모양이 이상함 (`items`가 없음 등)	공지가 없는 것으로 처리

세 파일이 함께 바뀌었습니다.

- **`home.html`**: 예시 공지 4개("69 tasks, one board" 등)를 지우고 빈 `<ol id="dev-notes">`와 안내 문구 자리(`#dev-notes-empty`), `+ View More` 자리(`#dev-notes-more`)를 두었습니다. 가짜 공지가 진짜처럼 보이는 일이 없게 하기 위해서입니다.
- **`home.css`**: 안내 문구 모양(`.dev-empty`)과, `hidden` 속성이 `display` 지정에 눌리지 않게 하는 규칙을 더했습니다.
- **`home.js`**: 두 함수를 구현했습니다.

2. 언제 쓰나

호출하는 곳	시점
`home.js` 맨 아래 `loadDevNotes();`	홈이 열릴 때 한 번
`formatNoteDate()`	`loadDevNotes()`가 카드마다 부르고, 다음 함수(98, View More 모달)도 씁니다

3. 코드

import: 새로 추가한 것은 없습니다 (`api`는 이미 가져와 있었습니다).

날짜 글자

```
export function formatNoteDate(isoTime) {
  const hasZone = /[zZ]$|[+-]\d\d:\d\d$/.test(isoTime);
  const date = new Date(hasZone ? isoTime : isoTime + 'Z');
  if (isNaN(date.getTime())) return { label: '', iso: '' };
  const pad = function (n) { return n < 10 ? '0' + n : String(n); };
  const y = date.getFullYear();
  const m = pad(date.getMonth() + 1);
  const d = pad(date.getDate());
  return { label: y + '.' + m + '.' + d, iso: y + '-' + m + '-' + d };
}
```

공지 불러오기

```
export async function loadDevNotes() {
  const list = document.getElementById('dev-notes');
  const empty = document.getElementById('dev-notes-empty');
  const more = document.getElementById('dev-notes-more');
  if (!list) return;

  const showMessage = function (text) {
    list.textContent = '';
    if (more) more.hidden = true;
    if (empty) { empty.textContent = text; empty.hidden = false; }
  };

  let data = null;
```

```
  try {
    data = await api.get('/dev-notes?limit=4', { auth: false });
  } catch (err) {
    showMessage('Notes are unavailable right now. Please try again later.');
    return;
  }
  const items = data && Array.isArray(data.items) ? data.items : [];
  if (items.length === 0) { showMessage('No notes yet. Check back soon.'); return; }

  list.textContent = '';
  for (const note of items) {
    const when = formatNoteDate(note.published_at);
    const item = document.createElement('li');
    item.className = 'dev-note';
    const time = document.createElement('time');
    if (when.iso) time.setAttribute('datetime', when.iso);
    time.textContent = when.label;
    const title = document.createElement('h3');
    title.className = 'dev-note__title';
    title.textContent = note.title;
    const excerpt = document.createElement('p');
    excerpt.className = 'dev-note__excerpt';
    excerpt.textContent = note.summary;
    item.appendChild(time);
    item.appendChild(title);
    item.appendChild(excerpt);
    list.appendChild(item);
  }
  if (empty) empty.hidden = true;
  if (more) more.hidden = false;
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export const loadDevNotes = async () => {
  const list = document.getElementById('dev-notes');
  if (!list) return;

  let items;
  try {
    ({ items = [] } = (await api.get('/dev-notes?limit=4', { auth: false })) ?? {});
  } catch {
    return showMessage('Notes are unavailable right now. Please try again later.');
  }
  if (!items.length) return showMessage('No notes yet. Check back soon.');

  list.replaceChildren(...items.map((note) => {
    const { label, iso } = formatNoteDate(note.published_at);
    const li = document.createElement('li');
    li.className = 'dev-note';
    li.append(
      Object.assign(document.createElement('time'), { textContent: label, dateTime: iso }),
      Object.assign(document.createElement('h3'), { className: 'dev-note__title', textContent: note.titl
e }),
      Object.assign(document.createElement('p'), { className: 'dev-note__excerpt', textContent: note.sum
mary }),
```

```
    );
    return li;
  }));
};
```

최신 문법	실제 코드	뜻
`?? {}`	`data && Array.isArray(data.items) ? data.items : []`	값이 `null`이나 `undefined`일 때 기본값을 씁니다
`catch { }`	`catch (err) { }`	쓰지 않는 오류 변수를 생략합니다
`({ items = [] } = …)`	`data.items`	객체에서 꺼내면서 기본값을 정합니다 (구조 분해)
`list.replaceChildren(...목록)`	`textContent = ''`와 `appendChild` 반복	자식을 한 번에 갈아 끼웁니다
`items.map(…)`	`for (const note of items)`	목록을 다른 목록으로 바꿉니다
`Object.assign(요소, { … })`	`요소.속성 = 값`을 줄마다	속성 여러 개를 한 번에 넣습니다
`li.append(a, b, c)`	`appendChild`를 세 번	여러 자식을 한 번에 붙입니다

문법 풀이

식	뜻
`api.get(경로, { auth: false })`	로그인 없이 읽는 API라서 토큰을 붙이지 않고, 토큰이 없다고 재발급을 시도하지도 않습니다
`Array.isArray(data.items)`	`items`가 목록인지 확인합니다
`document.createElement('li')`	새 HTML 요소를 만듭니다
`element.textContent = 글자`	글자를 **글자 그대로** 넣습니다. `<b>` 같은 글자가 있어도 태그로 해석하지 않습니다
`list.textContent = ''`	안의 내용을 모두 비웁니다 (다시 그릴 때 겹치지 않게)
`isoTime + 'Z'`	서버의 시각은 UTC이고 시간대 표시가 없어서(`2026-10-12T22:30:00`) 끝에 `Z`를 붙여 UTC로 읽게 합니다
`date.getFullYear()`, `getMonth()`, `getDate()`	이 컴퓨터의 시간대 기준 연, 월(0부터 시작), 일입니다 (`getUTC…`와 다릅니다)
`isNaN(date.getTime())`	읽을 수 없는 날짜인지 확인합니다

4. 흐름

1. 목록, 안내 문구, View More의 요소를 찾고, 목록이 없으면 끝냅니다.
2. 토큰 없이 공지 API를 부릅니다. 실패하면 오류 안내 문구를 보이고 끝냅니다.
3. 응답에서 `items` 목록을 꺼냅니다. 비어 있으면 "공지 없음" 문구를 보입니다.
4. 목록을 비우고 공지마다 카드를 만들어 붙입니다.
5. 안내 문구를 숨기고 `+ View More`를 보입니다.

5. 왜 이렇게

선택	이유
글자를 `textContent`로만 넣음	공지는 관리자가 쓰지만, 관리자 계정이 뚫리거나 실수로 `<script>`를 넣어도 홈 방문자에게 실행되지 않게 합니다 (시험 D04에서 `<img onerror>`로 확인했습니다)
인증 없이 부름 (`auth: false`)	홈은 로그인 전에도 보이는 화면입니다. 인증을 켜면 토큰이 없을 때 재발급을 먼저 시도해 요청이 한 번 더 나갑니다
예시 공지를 HTML에서 지움	불러오는 동안이나 실패했을 때 가짜 공지가 진짜 공지처럼 보이는 것을 막습니다
오류를 밖으로 던지지 않음	홈에는 공지 말고도 로그인 모달, 문의 폼 등이 있습니다. 공지 하나가 실패해도 나머지는 동작해야 합니다
실패와 "공지 없음"을 다른 문구로	사용자에게 "아직 글이 없음"과 "지금 못 불러옴"은 다른 상황입니다
날짜를 이 컴퓨터의 시간대로	같은 공지가 서울에서는 10월 13일, 로스앤젤레스에서는 10월 12일일 수 있습니다. 마이페이지와 같은 방식입니다 (P1-17)
날짜 계산을 `formatNoteDate`로 따로 뺌	다음 함수(98, View More 모달)도 같은 날짜 글자를 씁니다
`<time datetime="2026-10-13">`	사람은 `2026.10.13`으로 읽고, 기계(검색 로봇, 화면 낭독기)는 `datetime` 값을 읽습니다
다시 불러도 같은 결과	목록을 비우고 그리므로 두 번 불러도 카드가 겹치지 않습니다

6. 시험 (실제 서버와 실제 DB, 헤드리스 Chrome, 16개)

공지 9개를 DB에 넣고 시험했습니다 (공개 5개, 초안, 예약 중, 삭제, 그리고 제목과 요약에 `<b>`, `<img onerror>`가 들어간 것). 시험 데이터는 P1-23이 끝날 때 지웁니다.

#	한 일	결과
D01	로그인하지 않고 홈을 엶	최신 4개가 카드로, 최신순 ✓
D02	초안, 예약 중, 삭제한 공지	카드에 없음 ✓
D03	카드의 요약과 제목 요소	요약이 들어가고 제목은 `<h3>` ✓
D04	제목과 요약에 `<b>`, `<img onerror>`	글자로 보이고 태그가 되지 않음 ✓
D05	공지가 있을 때	안내 문구는 숨김, `+ View More`는 보임 ✓
D06	`loadDevNotes`가 보내는 요청	공지 API 한 번뿐, `/auth/refresh` 없음 ✓
D07	시간대를 서울로 (UTC 22:30 공지)	`2026.10.13` (다음 날), `datetime="2026-10-13"` ✓
D08	시간대를 로스앤젤레스로	같은 공지가 `2026.10.12` ✓
D09	`formatNoteDate`에 여러 값	`Z` 없는 값은 UTC로, `Z`와 `+09:00`은 그대로, 읽을 수 없으면 빈 글자 ✓
D10	응답이 빈 목록	카드 없음, "No notes yet...", View More 숨김 ✓
D11	서버 오류(500)	"Notes are unavailable...", View More 숨김 ✓
D12	네트워크 끊김	같은 오류 안내 ✓
D13	응답에 `items`가 없음	공지 없음으로 처리, 오류 없음 ✓
D14	두 번 연속 호출	카드 4개 그대로(겹치지 않음), 안내 문구 숨김 ✓

#	한 일	결과
D15	목록 요소가 없음	오류 없이 끝남 ✓
D16	폭 390	가로로 넘치지 않음 ✓

16개 모두 통과했습니다. 처음 돌렸을 때 D06이 실패했는데, 홈이 열릴 때 머리글의 로그인 상태 확인(`initAuthState`)이 `/auth/refresh`를 부르는 것을 `loadDevNotes`의 요청으로 잘못 센 시험의 문제였습니다. `loadDevNotes`만 따로 불러서 요청을 세도록 시험을 고쳤습니다.

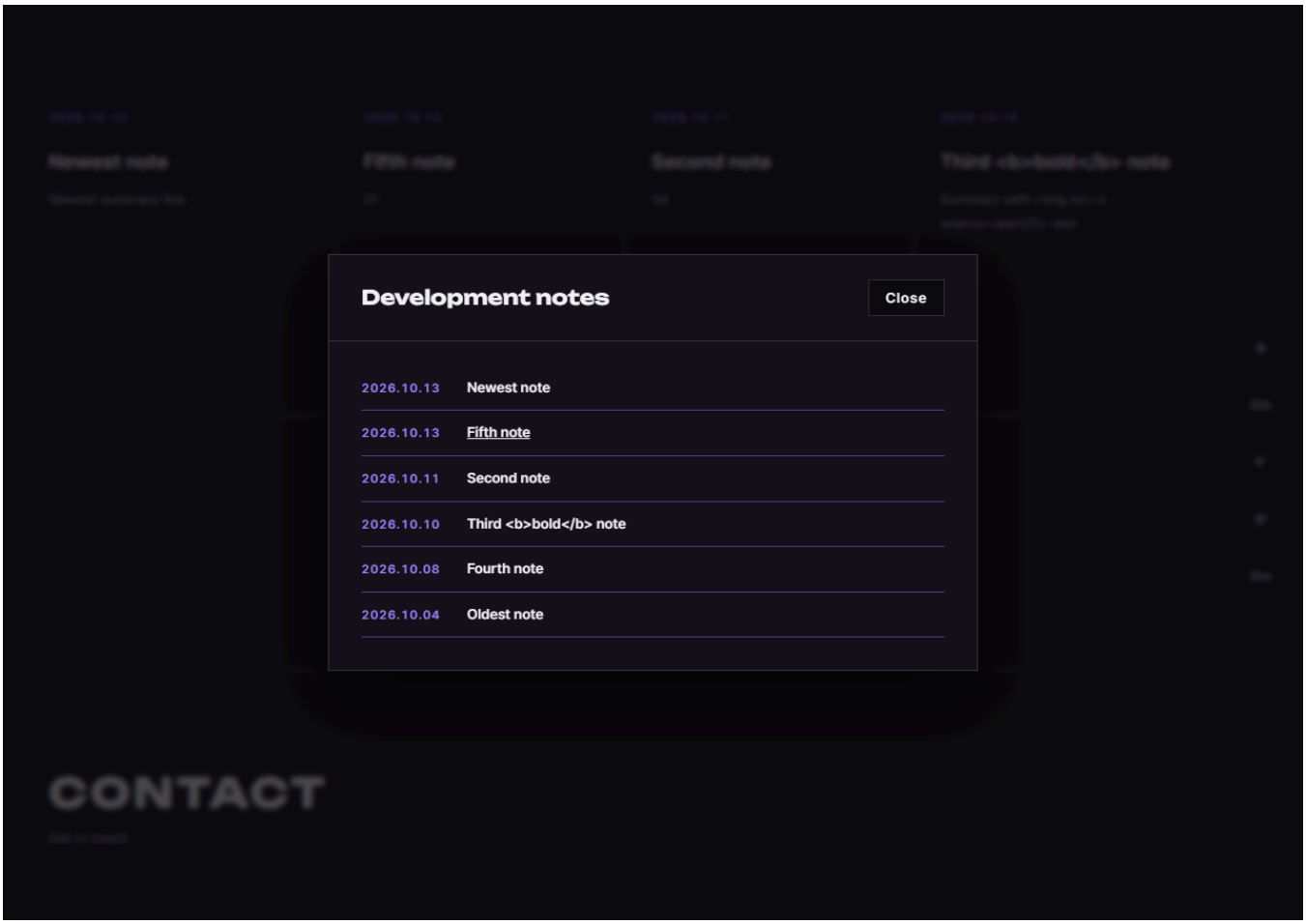
JS 98 — `initDevNotesModal()` (+ View More 공지 모달: 목록과 상세)

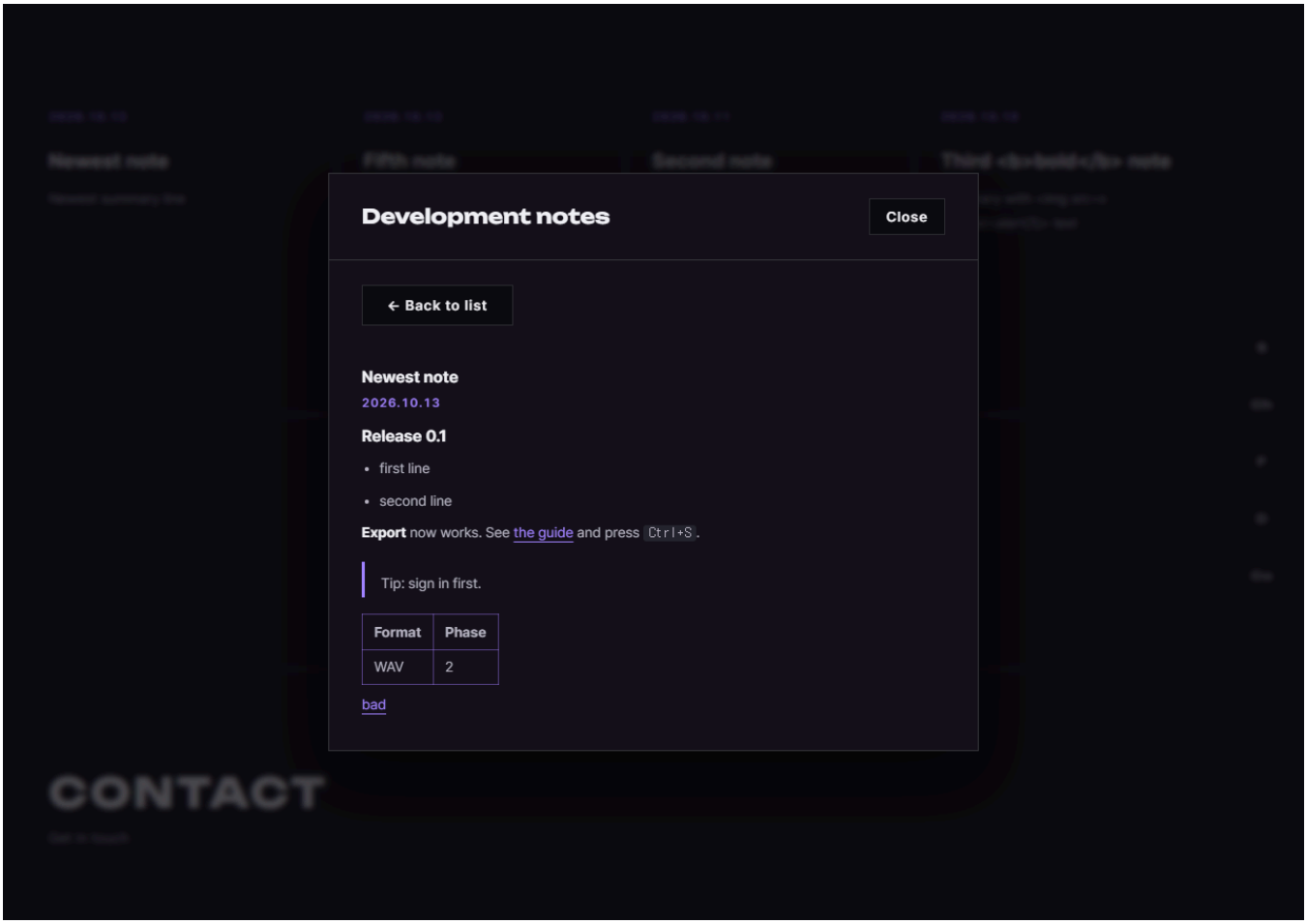
항목	내용
파일	`studionx/frontend/static/js/pages/home.js`, `home.html`, `static/css/pages/home.css`
작업	P1-23 화면 마무리: `+ View More`를 누르면 공지 전체 목록을 보여 주고, 하나를 누르면 Markdown 본문을 안전하게 보여 주는 모달 (PRD 01 FR-114)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	99 `markdownToFragment()` · 100 `initSideNavActiveState()`, `initMobileMenuClose()`, `initIntro()` (다음)

1. 하는 일

지금까지 "The full archive … will be listed here."라는 안내 문구뿐이던 모달에 두 화면을 넣었습니다.

화면	내용
목록	공개된 공지를 최신순으로 최대 50개, 날짜(보라색)와 제목 한 줄씩
상세	`← Back to list` 버튼, 제목, 날짜, 본문(Markdown을 서식으로 변환)





동작	내용
`+ View More`를 누름	모달이 열리고(HTML의 `command` 속성이 엶), **그때마다** `GET /dev-notes?limit=50`으로 목록을 새로 불러 새 공지가 반영됩니다
목록에서 공지를 누름	`GET /dev-notes/{id}`로 본문을 받아 상세 화면으로 바꿉니다. 본문은 99번 `markdownToFragment()`로만 넣습니다
`Back to list`	목록 화면으로 돌아갑니다. 목록은 다시 부르지 않고, 방금 본 공지에 포커스를 돌려줍니다
모달을 닫음(`Close`, `Esc`)	다음에 열 때는 항상 목록 화면부터 보입니다
공지가 하나도 없음	"No notes yet. Check back soon."
목록을 불러오지 못함	"Notes are unavailable right now. Please try again later."
상세에서 404 (그 사이 비공개나 삭제됨)	"This note is no longer available." 와 `Back to list`
상세를 불러오지 못함 (네트워크 등)	"The note could not be loaded. Please try again later."

함께 바뀐 파일은 아래와 같습니다.

- **`home.html`**: 모달 안을 `#note-list-view`(목록)와 `#note-detail-view`(상세, `Back to list` 버튼, 제목, 날짜, 본문, 안내 문구)로 바꿨습니다.
- **`home.css`**: 목록과 상세 모양, 본문 서식(제목 크기를 모달 제목보다 크지 않게 맞춤, 표와 코드 상자가 가로로 넘치지 않게 함)을 더했습니다.
- **`home.js`**: 이 함수와 맨 아래의 `initDevNotesModal();` 호출을 더했고, `markdownToFragment`를 가져옵니다.

2. 언제 쓰나

호출하는 곳	시점
`home.js` 맨 아래 `initDevNotesModal();`	홈이 열릴 때 한 번 (이벤트 처리기를 연결)

3. 코드

import (이번에 추가한 것)

```
import { markdownToFragment } from '../core/markdown.js';
```

함수 (줄이 길어 핵심 구조만 옮깁니다. 전체는 `home.js`에 있습니다)

```
export function initDevNotesModal() {
  const modal = document.getElementById('dev-notes-modal');
  const more = document.querySelector('#dev-notes-more button');
  /* listView, detailView, list, listEmpty, back, title, date, body, detailEmpty 요소 찾기 */
  if (!modal || !list || !listView || !detailView) return;

  let requestId = 0;       // 요청마다 번호를 올려서, 늦게 온 이전 응답을 버림
  let lastOpenedId = null; // Back to list 때 포커스를 돌려줄 공지

  const showList = function () {
    requestId += 1;
    detailView.hidden = true;
    listView.hidden = false;
    if (lastOpenedId !== null) {
      const item = list.querySelector('[data-note-id="' + lastOpenedId + '"]');
      if (item) item.focus();
    }
  };

  const openNote = async function (noteId) {
    requestId += 1;
    const mine = requestId;
    lastOpenedId = noteId;
    listView.hidden = true;
    detailView.hidden = false;
    title.textContent = 'Loading…';
    body.textContent = '';
    back.focus();

    let note = null;
    let failure = '';
    try {
      note = await api.get('/dev-notes/' + encodeURIComponent(noteId), { auth: false });
    } catch (err) {
      failure = err && err.status === 404
        ? 'This note is no longer available.'
        : 'The note could not be loaded. Please try again later.';
    }
    if (mine !== requestId) return;   // 그 사이 다른 공지를 눌렀거나 목록으로 돌아감
```

```
    if (!note) { title.textContent = ''; detailEmpty.textContent = failure; detailEmpty.hidden = false;
return; }
    const when = formatNoteDate(note.published_at);
    title.textContent = note.title;
    date.textContent = when.label;
    body.appendChild(markdownToFragment(note.content));
    title.focus();
  };

  const loadList = async function () { /* 같은 방식: 번호표, 불러오기, 줄마다 버튼 만들기 */ };

  list.onclick = function (event) {
    const button = event.target.closest('[data-note-id]');
    if (button) openNote(Number(button.getAttribute('data-note-id')));
  };
  back.onclick = showList;
  if (more) more.onclick = loadList;
  modal.onclose = function () {
    requestId += 1;              // 닫은 뒤에 도착하는 응답은 버림
    detailView.hidden = true;
    listView.hidden = false;
  };
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export const initDevNotesModal = () => {
  const $ = (id) => document.getElementById(id);
  const modal = $('dev-notes-modal');
  if (!modal) return;
  let requestId = 0;

  const openNote = async (noteId) => {
    const mine = ++requestId;
    try {
      const note = await api.get(`/dev-notes/${encodeURIComponent(noteId)}`, { auth: false });
      if (mine !== requestId) return;
      $('note-detail-title').textContent = note.title;
      $('note-detail-body').replaceChildren(markdownToFragment(note.content));
    } catch (err) {
      if (mine !== requestId) return;
      showDetailMessage(err?.status === 404 ? 'This note is no longer available.' : 'The note could not
be loaded.');
    }
  };
  $('note-list').addEventListener('click', ({ target }) => {
    const id = target.closest('[data-note-id]')?.dataset.noteId;
    if (id) openNote(Number(id));
  });
};
```

최신 문법	실제 코드	뜻
`++requestId`	`requestId += 1;` 다음 줄에서 읽기	올리면서 바로 값을 씁니다
`err?.status`	`err && err.status`	값이 없을 때 안전하게 꺼냅니다 (옵셔널 체이닝)

최신 문법	실제 코드	뜻
`` `/dev-notes/${id}` ``	`'/dev-notes/' + id`	글자에 값을 끼워 넣습니다 (템플릿 문자열)
`({ target }) =>`	`function (event)`와 `event.target`	화살표 함수와 구조 분해입니다
`.dataset.noteId`	`getAttribute('data-note-id')`	`data-` 속성을 객체처럼 읽습니다
`replaceChildren(조각)`	`textContent = ''`와 `appendChild`	자식을 한 번에 갈아 끼웁니다

문법 풀이

식	뜻
`requestId` (번호표)	요청을 시작할 때마다 올립니다. 응답이 돌아왔을 때 `mine !== requestId`이면 그 사이 다른 요청이나 닫기가 있었다는 뜻이라 **그 응답을 버립니다**. 느린 응답이 나중에 와서 화면을 덮어쓰는 것을 막습니다
`element.hidden = true/false`	요소를 숨기거나 보입니다 (CSS에 `[hidden]` 규칙을 함께 두었습니다)
`event.target.closest('[data-note-id]')`	누른 곳에서 위로 올라가며 `data-note-id`가 있는 가장 가까운 요소(공지 버튼)를 찾습니다. 줄 안의 날짜나 제목 글자를 눌러도 같은 버튼을 찾습니다
`list.onclick = …`	목록에 클릭 처리기를 **하나** 둡니다. `addEventListener`와 달리 다시 호출해도 처리기가 쌓이지 않고 덮어씁니다
`modal.onclose`	`<dialog>`가 닫힐 때(버튼, `Esc` 모두) 불립니다
`encodeURIComponent(noteId)`	주소에 넣어도 안전한 글자로 바꿉니다
`title.focus()`	화면 낭독기가 새 내용(제목)을 읽도록 포커스를 옮깁니다. 제목에 `tabindex="-1"`을 두어 코드로만 포커스를 받게 했습니다

4. 흐름

1. 홈이 열릴 때 요소를 찾아 처리기를 연결합니다. 필요한 요소가 없으면 아무것도 하지 않습니다.
2. `+ View More`를 누르면 모달이 열리고 `loadList`가 목록을 불러옵니다.
3. 공지를 누르면 `openNote`가 상세 화면으로 바꾸고 본문을 불러 그립니다. 응답 전에는 "Loading…"입니다.
4. `Back to list`는 목록을 다시 불러오지 않고 목록 화면으로만 돌아갑니다.
5. 모달이 닫히면 번호표를 올려서 늦게 오는 응답을 버리고, 다음에 열 때를 위해 목록 화면으로 되돌립니다.

5. 왜 이렇게

선택	이유
같은 모달 안에서 목록과 상세를 번갈아 보여 줌	모달 위에 모달을 또 띄우면 `Esc`와 포커스가 복잡해집니다. 화면 두 개를 숨기고 보이는 쪽이 단순합니다
열 때마다 목록을 새로 불러옴	관리자가 새 공지를 쓰거나 지웠을 때, 홈을 새로 고치지 않아도 다음에 열면 반영됩니다
`Back to list`는 목록을 다시 안 부름	방금 본 목록 그대로이고, 방금 본 공지에 포커스가 돌아가서 이어서 읽기 편합니다
번호표(`requestId`)로 늦은 응답을 버림	공지 A를 누르고 응답이 오기 전에 B를 누르면, 느린 A가 나중에 와서 B 화면을 덮어쓰는 문제가 생깁니다 (시험 G14, G15)
목록과 상세 모두 인증 없이(`auth: false`) 부름	홈은 로그인 전에도 보이는 화면이고, 공지는 공개 정보입니다 (97번과 같음)

선택	이유
본문은 `markdownToFragment`로만 넣음	안전 처리가 그 함수 안에 고정되어 있어서, 이 함수에서 빠뜨릴 수 없습니다
404를 별도 문구로	목록을 본 뒤에 관리자가 그 공지를 지웠거나 비공개로 바꿀 수 있습니다. "불러오지 못함"과 다른 상황입니다
`onclick`을 씀 (`addEventListener` 아님)	마이페이지에서와 같은 이유입니다. 두 번 호출해도 처리기가 겹치지 않습니다
목록은 최대 50개	API의 `limit` 최대값입니다. 공지가 50개를 넘는 때의 쪽 나누기는 필요해질 때 더합니다
제목에 포커스, 목록에는 항목에 포커스	키보드와 화면 낭독기 사용자가 화면이 바뀐 것을 알 수 있게 합니다

6. 시험 (실제 서버와 실제 DB, 마우스와 키보드로 조작, 23개)

공지 9개(공개 6, 초안, 예약 중, 삭제 각 1)를 DB에 두고 시험했습니다. `Newest note`의 본문에는 서식(제목, 목록, 굵게, 링크, 코드, 인용, 표)과 공격 글(`<script>`, `<img onerror>`, `javascript:` 링크)을 넣었습니다.

#	한 일	결과
G01	`+ View More`를 마우스로 누름	모달이 열리고 목록 화면 ✓
G02	목록	공개 6개가 최신순, 초안, 예약 중, 삭제는 없음 ✓
G03	공지를 누름	상세 화면(목록은 숨김), 제목과 날짜 ✓
G04	상세의 본문	제목, 목록, 굵게, 링크, 코드, 인용, 표가 서식으로 보임 ✓
G05	본문의 공격 글	실행되지 않고(`__xss`가 0) `script`, `img`가 남지 않음 ✓
G06	본문의 링크	허용된 링크는 새 창과 `noopener noreferrer`, 위험한 링크는 `href` 없음 ✓
G07	상세로 감	제목에 포커스 ✓
G08	`Back to list`	목록 화면, 방금 본 공지에 포커스 ✓
G09	서버 요청 횟수	목록 1번, 상세 1번 (돌아갈 때 다시 안 부름) ✓
G10, G11	`Close`로 닫고 다시 열기	닫힘, 다시 열면 목록 화면이고 목록을 새로 불러옴(2번째) ✓
G12, G13	상세에서 `Esc`로 닫고 다시 열기	닫힘, 다시 열면 목록 화면부터 ✓
G14	공지 A 응답이 1.2초 늦는 상태에서 A를 누르고 바로 목록으로 가서 B를 누름	늦게 온 A가 화면을 덮지 않고 B가 유지됨 ✓
G15	공지를 누르고 응답이 오기 전에 모달을 닫음	늦게 온 응답이 반영되지 않고 목록 화면 유지 ✓
G16, G17	목록을 본 뒤 그 공지를 삭제하고 누름	"This note is no longer available.", 제목은 비고 `Back to list`가 됨 ✓
G18	목록 서버 오류(500)	"Notes are unavailable..." ✓
G19	목록이 비어 있음	"No notes yet. Check back soon." ✓
G20	서버가 회복된 뒤 다시 열기	목록이 다시 나오고 문구가 사라짐 ✓
G21	상세에서 네트워크 끊김	"The note could not be loaded..." ✓
G22	목록 제목에 `<b>` 글자	태그가 되지 않고 글자로 보임 ✓
G23	폭 390	모달이 화면 안에 들어오고 표 포함 본문이 가로로 넘치지 않음 ✓

23개 모두 통과했습니다. 시험 중에 두 가지 시험 쪽 문제가 있었습니다. 처음에는 마우스 클릭이 모달을 못 열었는데, 사이트 CSS의 부드러운 스크롤 때문에 스크롤이 끝나기 전에 좌표로 눌러서였고(시험을 "즉시 스크롤"로 고침), 시험 공지의 본문이 줄바꿈 없는 한 줄로 저장되어 서식이 안 보였던 것은 시험 데이터를 넣는 스크립트의 문제였습니다. 두 경우 모두 앱 코드는 고치지 않았습니다.

JS 99 — `markdownToFragment()` (Markdown을 안전한 HTML로)

항목	내용
파일	`studionx/frontend/static/js/core/markdown.js` (새 파일), `static/vendor/marked/`, `static/vendor/dompurify/` (새 폴더)
작업	P1-23 화면 마무리: 관리자가 Markdown으로 쓴 공지 본문을 방문자 화면에 **안전하게** 보여 주는 변환 함수 (PRD 01 FR-114)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	97 `loadDevNotes()` · 98 View More 모달 (다음. 번호는 순서와 달리 99를 먼저 만들었습니다: 보안이 걸린 이 함수를 먼저 시험하기 위해서입니다)

1. 하는 일

Markdown 글을 받아 **안전하게 걸러 낸 DOM 조각**(`DocumentFragment`)으로 돌려줍니다. 다음 함수(98번, 공지 모달)가 이 조각을 화면에 붙입니다. 사용자가 C안(라이브러리)을 골랐습니다 (2026-10-13, 목업: `docs/page_mockups/dev_notes_modal/`).

입력(Markdown)	결과
`# 제목`, `- 목록`, `**굵게**`, `` `코드` ``, `> 인용`, 표(`| a | b |`)	제목, 목록, 굵은 글자, 코드, 인용, 표 서식
`[글자](https://…)`	링크. **새 창으로 열리고** `rel="noopener noreferrer"`가 붙습니다
줄바꿈(Enter 한 번)	줄바꿈(` `)
`<script>`, `<iframe>`, `<form>`, `<style>`, `<svg>`, `<object>`	지워짐
`![그림](주소)`, `<img>`	지워짐 (외부 그림은 방문 기록이 새는 길이라 막습니다)
`onclick`, `onerror`, `ontoggle` 같은 `on…` 속성, `style` 속성	지워짐
`javascript:`, `data:` 주소의 링크, 상대 경로와 `#` 링크	주소(`href`)가 지워져 그냥 글자만 남음. `http`, `https`, `mailto`만 링크로 남습니다
글이 아닌 값(`null`, 숫자, 빈 글자)	빈 조각 (오류 없음)

2. 새로 들어온 도구와 SweetAlert2

도구	버전, 라이선스	위치	하는 일
marked	18.1.1, MIT	`static/vendor/marked/marked.esm.js`	Markdown → HTML
DOMPurify	3.4.17, MPL-2.0 또는 Apache-2.0	`static/vendor/dompurify/purify.es.mjs`	위험한 HTML 제거

- SweetAlert2(FIX-01)와 **같은 방식**입니다. npm 패키지의 ES 모듈 파일을 `static/vendor/`에 넣었고, 외부 CDN은 쓰지 않습니다. 라이선스 파일과 버전을 적은 `VERSION.txt`를 함께 두었습니다. 원본에서 바꾼 것은 파일 끝의 `sourceMappingURL` 주석 한 줄을 뺀 것뿐입니다 (브라우저가 없는 `.map` 파일을 찾느라 404를 내지 않게 하려는 것입니다).

- **SweetAlert2에는 영향이 없습니다.** 새 도구는 ES 모듈이라 전역 변수를 만들지 않고, 다른 폴더(`vendor/marked`, `vendor/dompurify`)에 있어 `vendor/sweetalert2` 파일은 그대로입니다. 시험 S01 ~ S03에서 확인했습니다 (§6).

3. 언제 쓰나

호출하는 곳	시점
98 공지 모달 (다음)	공지 상세에서 본문(`content`)을 보여 줄 때

4. 코드

```
import { marked } from '../../vendor/marked/marked.esm.js';
import DOMPurify from '../../vendor/dompurify/purify.es.mjs';

export function markdownToFragment(markdown) {
  if (typeof markdown !== 'string' || markdown === '') return document.createDocumentFragment();

  const html = marked.parse(markdown, { gfm: true, breaks: true, async: false });
  const fragment = DOMPurify.sanitize(html, {
    RETURN_DOM_FRAGMENT: true,
    USE_PROFILES: { html: true },
    FORBID_TAGS: ['style', 'form', 'input', 'button', 'textarea', 'select', 'option', 'img', 'picture',
      'source', 'video', 'audio', 'iframe', 'object', 'embed', 'link', 'meta', 'base'],
    FORBID_ATTR: ['style', 'srcset'],
    ALLOWED_URI_REGEXP: /^(?:https?:|mailto:)/i,
    ALLOW_DATA_ATTR: false,
  });

  const links = fragment.querySelectorAll('a');
  for (const link of links) {
    if (!link.hasAttribute('href')) continue;
    link.setAttribute('target', '_blank');
    link.setAttribute('rel', 'noopener noreferrer');
  }
  return fragment;
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export const markdownToFragment = (markdown) => {
  if (typeof markdown !== 'string' || !markdown) return document.createDocumentFragment();
  const fragment = DOMPurify.sanitize(marked.parse(markdown, OPTIONS), PURIFY_OPTIONS);
  fragment.querySelectorAll('a[href]').forEach((a) => a.setAttribute('target', '_blank') || a.setAttribu
te('rel', 'noopener noreferrer'));
  return fragment;
};
```

최신 문법	실제 코드	뜻
`querySelectorAll('a[href]')`	`querySelectorAll('a')`와 `hasAttribute('href')` 검사	`href`가 있는 링크만 고릅니다
`.forEach((a) => …)`	`for (const link of links)`	목록을 하나씩 처리합니다

최신 문법	실제 코드	뜻
`!markdown`	`markdown === ''`	빈 글자를 거짓으로 봅니다

문법 풀이

식	뜻
`marked.parse(글, { gfm: true, breaks: true, async: false })`	Markdown을 HTML 글자로 바꿉니다. `gfm`은 표 같은 확장 문법, `breaks`는 Enter 한 번도 줄바꿈, `async: false`는 결과를 바로 돌려받는다는 뜻입니다
`DOMPurify.sanitize(html, 설정)`	HTML에서 위험한 부분을 지웁니다
`RETURN_DOM_FRAGMENT: true`	글자가 아니라 DOM 조각으로 돌려줍니다. 글자(`innerHTML`)로 다시 바꿔서 넣으면 브라우저가 한 번 더 해석해서 우회될 수 있어서(변이 XSS) 조각 그대로 붙입니다
`USE_PROFILES: { html: true }`	일반 HTML 태그만 허용하는 기본 묶음입니다 (SVG, MathML 제외)
`FORBID_TAGS`, `FORBID_ATTR`	기본 허용 범위에서 추가로 막을 태그와 속성입니다
`ALLOWED_URI_REGEXP: /^(?:https?:|mailto:)/i`	링크 주소가 `http:`, `https:`, `mailto:`로 시작할 때만 남깁니다 (`i`는 대소문자 무시)
`ALLOW_DATA_ATTR: false`	`data-…` 속성을 막습니다
`document.createDocumentFragment()`	비어 있는 DOM 조각입니다
`link.hasAttribute('href')`	주소가 있는 링크인지 봅니다. 위험한 주소라서 지워졌으면 `href`가 없습니다

5. 흐름

1. 글이 아니거나 비어 있으면 빈 조각을 돌려줍니다.
2. `marked`가 Markdown을 HTML로 바꿉니다.
3. `DOMPurify`가 그 HTML을 걸러서 DOM 조각으로 만듭니다.
4. 남은 링크마다 새 창으로 열고 `rel`을 붙입니다.
5. 조각을 돌려줍니다. 부르는 쪽은 `appendChild`로 붙이면 됩니다.

6. 왜 이렇게

선택	이유
변환 → 안전 처리 → 링크 손질을 **한 함수에 고정**	안전 처리를 부르는 쪽이 빠뜨릴 수 없습니다. 본문을 HTML로 넣는 곳은 이 함수 하나뿐이어야 합니다
허용 목록 방식(필요한 것만 남김)	막을 것을 나열하면 빠뜨린 것이 구멍이 됩니다. DOMPurify의 기본 허용 범위에서 필요 없는 것을 더 줄였습니다
그림(`img`, `svg`, `video`)도 막음	공지에 그림이 필요하면 나중에 허용 주소를 정해 열 수 있습니다. 지금은 외부 주소로 방문자의 접속 기록이 새는 것을 막는 쪽을 택했습니다
`style` 속성과 태그를 막음	글이 화면 전체를 가리거나(`position: fixed`) 숨기는(`display: none`) 장난을 막습니다
링크 주소를 `http`, `https`, `mailto`로 제한	`javascript:`, `data:` 주소가 눌렀을 때 실행되는 것을 막습니다
새 창 + `noopener noreferrer`	열린 사이트가 우리 창을 조작하지 못하게 하고, 우리 주소를 넘겨주지 않습니다
`breaks: true`	관리자는 입력칸에서 Enter로 줄을 나눕니다. Markdown 규칙(빈 줄이 있어야 문단)을 몰라도 쓴 대로 보입니다

선택	이유
조각(`DocumentFragment`)으로 돌려줌	글자로 돌려주면 `innerHTML`로 넣게 되는데, 걸러 낸 결과를 다시 글자로 만들었다가 해석하는 과정에서 우회가 가능합니다 (변이 XSS)
글 안의 `#` 제목은 태그를 바꾸지 않고 CSS로 크기를 맞춤	모달 제목과 공지 제목보다 크게 보이지 않게 합니다 (98번에서 CSS)

7. 시험 (실제 서버, 헤드리스 Chrome, 26개)

브라우저에서 변환한 결과를 **실제 문서에 붙여서**(실행될 수 있는 상황) 확인했습니다. 공격 글이 실행되면 `window.__xss`가 1로 바뀌도록 해 두었습니다.

묶음	시험	결과
서식	제목, 목록, 굵게, 링크, 코드, 인용, 표가 모두 서식으로 바뀜	✓
링크	새 창, `rel="noopener noreferrer"`	✓
줄바꿈	Enter 한 번이 ` `	✓
공격 글 16종	`<script>`, `<img onerror>`, 마크다운 그림, `javascript:` 링크, 대소문자를 섞은 `JaVaScRiPt:`, 줄바꿈으로 깨뜨린 `java
script:`, `data:` 링크, HTML `<a href="javascript:…" onclick>`, `<svg onload>`, `<iframe>`, `<style>`와 `style` 속성, `<form>`과 입력칸, `<details ontoggle>`, `<math>` 변이 XSS, 링크 글자 안의 `<img onerror>`, `<object>`와 `<embed>`	모두 실행되지 않음 (`__xss`가 0), `on…` 속성 0개, 위험한 태그와 주소가 남지 않음 ✓
일반	안전한 글자는 남고 `<script>`만 사라짐	✓
일반	`mailto:`, `http://`만 링크로 남고 상대 경로와 `#` 링크는 `href`가 지워짐	✓
이상한 입력	`null`, `undefined`, 숫자, 객체, 배열, 빈 글자를 넣으면 빈 조각	✓
성능	3000줄(링크 3000개)을 1초 안에 처리	✓
SweetAlert2	두 도구를 불러온 뒤에도 **새 전역 변수가 생기지 않음** (`window.marked`, `window.DOMPurify` 없음)	✓
SweetAlert2	`showAlert`로 알림창이 그대로 뜸	✓
SweetAlert2	`vendor/sweetalert2` 파일이 그대로 제공됨 (200)	✓

26개 모두 통과했습니다. 처음 돌렸을 때 5개(`javascript:`, `data:` 링크 등)가 실패로 나왔는데, 위험한 주소의 `href`가 **지워지는 것이 의도된 결과**인데 시험이 "`href`가 있어야 한다"고 잘못 확인한 시험의 문제였습니다. 이때 함수도 하나 고쳤습니다. `href`가 지워진 링크에도 `target`과 `rel`이 붙고 있었는데, 주소가 없는 링크는 글자로만 남도록 건너뛰게 했습니다.

JS 100 — `initIntro()`, `initSideNavActiveState()`, `initMobileMenuClose()` (홈 화면 마무리 세 가지)

항목	내용
파일	`studionx/frontend/static/js/pages/home.js`, `home.html`
작업	P1-23 화면 마무리: 홈에 비어 있던 세 함수를 채움. 첫 접속 로딩을 처음에만 보이기, 세로 앵커의 현재 섹션 표시, 모바일 메뉴 닫기
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	98 `initDevNotesModal()` · 101 `privacy.js` 목차 표시와 정리 (다음)

작은 함수 세 개라서 한 문서에 함수별로 나눠 적었습니다.

A. `initIntro()` (첫 접속 로딩은 같은 탭에서 처음에만)

A-1. 하는 일

홈에 들어오면 약 1초짜리 로딩("Crescent")이 재생됩니다. 지금까지는 **홈에 올 때마다** 재생됐습니다. 이제 같은 탭에서 이미 봤다면 두 번째부터는 보이지 않습니다.

상황	결과
이 탭에서 처음	로딩이 재생되고, 탭에 `snx-intro = seen`을 기록
같은 탭에서 두 번째부터	로딩이 **전혀 보이지 않음**, 첫 화면 애니메이션은 원래 시간대로
새 탭이나 탭을 닫았다 열기	기록이 없으므로 다시 재생
저장소를 못 쓰는 환경(사생활 보호 모드 등)	오류 없이 열리고 매번 재생

A-2. 코드

`home.html`의 `<head>` (CSS가 적용되기 전에 실행되는 짧은 검사)

```
<script>try { if (sessionStorage.getItem('snx-intro') === 'seen') document.documentElement.dataset.intro = 'seen'; } catch (e) {}</script>
```

`home.js`

```
export function initIntro() {
  try {
    if (sessionStorage.getItem('snx-intro') === 'seen') {
      document.documentElement.dataset.intro = 'seen';
    } else {
      sessionStorage.setItem('snx-intro', 'seen');
    }
  } catch (err) {
    // 저장소를 못 쓰면 기록하지 못함 → 그냥 매번 보여 줌
  }
}
```

최신 문법으로 쓰면 (공부용)

```
export const initIntro = () => {
  try {
    if (sessionStorage.getItem('snx-intro') === 'seen') document.documentElement.dataset.intro = 'seen';
    else sessionStorage.setItem('snx-intro', 'seen');
  } catch {}
};
```

최신 문법	실제 코드	뜻
`catch {}`	`catch (err) { … }`	쓰지 않는 오류 변수를 생략합니다

식	뜻
`sessionStorage`	이 탭이 열려 있는 동안만 값을 기억하는 저장소입니다 (탭을 닫으면 사라짐)
`document.documentElement.dataset.intro = 'seen'`	`<html data-intro="seen">`을 답니다. `intro.css`가 이 속성이 있으면 로딩을 숨깁니다
`try … catch`	저장소가 막힌 환경에서는 접근만 해도 오류가 나므로 감쌉니다

A-3. 왜 이렇게

선택	이유
`<head>`에도 같은 검사를 한 줄 둠	`home.js`는 머리글과 바닥글을 끼운 뒤(서버에서 받아 옴)에야 실행되어 너무 늦습니다. 그때 숨기면 로딩이 잠깐 보였다가 사라지는 **깜빡임**이 생깁니다. `<head>`의 검사는 CSS가 적용되기 전에 실행되어 처음부터 숨깁니다 (시험 I05)
기록은 `initIntro`가 남김	기록하는 곳을 한 곳으로 합니다. `<head>`는 읽기만 합니다
`sessionStorage`(탭 단위)	"첫 접속"을 탭 단위로 봅니다. 홈의 로고를 눌러 다시 오거나 다른 페이지를 다녀와도 다시 보이지 않고, 새로 방문할 때만 봅니다
`initIntro`에도 읽는 검사가 있음	나중에 보안 정책으로 인라인 스크립트가 막히더라도, 깜빡임은 생기지만 로직은 동작하도록 하는 대비책입니다
저장소 접근을 `try`로 감쌈	사생활 보호 모드에서 `sessionStorage`가 오류를 내도 홈은 열려야 합니다

B. `initSideNavActiveState()` (세로 앵커에서 지금 보는 섹션 표시)

B-1. 하는 일

화면 오른쪽의 세로 앵커(`S`, `Ch`, `F`, `D`, `Co`)에서, **지금 화면 가운데에 있는 섹션**의 링크에 `aria-current="true"`를 달아 보라색으로 표시합니다 (CSS는 이미 있었습니다).

스크롤 위치	표시되는 링크
맨 위의 첫 화면(hero)	없음
Service, Character, Feature, Development, Contact 섹션이 화면 가운데에 옴	그 섹션의 링크 하나만
맨 아래	마지막 섹션(Contact)
두 섹션의 경계	하나 이하 (위쪽 것)

B-2. 코드

```
export function initSideNavActiveState() {
  const links = document.querySelectorAll('.side-nav__link');
  if (links.length === 0 || typeof IntersectionObserver === 'undefined') return;

  const targets = [];   // { id, link, section, inside }
  for (const link of links) {
    const href = link.getAttribute('href') || '';
    if (href.charAt(0) !== '#') continue;
    const section = document.getElementById(href.slice(1));
    if (section) targets.push({ id: href.slice(1), link: link, section: section, inside: false });
  }
  if (targets.length === 0) return;

  const refresh = function () {
    let current = null;
    for (const target of targets) {
      if (target.inside) { current = target; break; }
    }
    for (const target of targets) {
      if (target === current) target.link.setAttribute('aria-current', 'true');
      else target.link.removeAttribute('aria-current');
    }
  };

  const observer = new IntersectionObserver(function (entries) {
    for (const entry of entries) {
      for (const target of targets) {
        if (target.section === entry.target) target.inside = entry.isIntersecting;
      }
    }
    refresh();
  }, { rootMargin: '-50% 0px -50% 0px', threshold: 0 });

  for (const target of targets) observer.observe(target.section);
}
```

최신 문법으로 쓰면 (공부용)

```
export const initSideNavActiveState = () => {
  const targets = [...document.querySelectorAll('.side-nav__link')]
    .map((link) => ({ link, section: document.getElementById(link.getAttribute('href')?.slice(1)), insid
e: false }))
    .filter((t) => t.section);
  if (!targets.length || !('IntersectionObserver' in window)) return;
  const observer = new IntersectionObserver((entries) => {
    entries.forEach(({ target: el, isIntersecting }) => { targets.find((t) => t.section === el).inside =
isIntersecting; });
    const current = targets.find((t) => t.inside);
    targets.forEach((t) => t.link.toggleAttribute('aria-current', t === current));
  }, { rootMargin: '-50% 0px -50% 0px' });
  targets.forEach((t) => observer.observe(t.section));
};
```

최신 문법	실제 코드	뜻
`?.slice(1)`	`	
`.map(...).filter(...)`	`for` 반복과 `if`	목록을 바꾸고 거릅니다
`{ link, section }`	`{ link: link, section: section }`	이름이 같으면 줄여 씁니다
`toggleAttribute`	`setAttribute`와 `removeAttribute`	속성을 켜고 끕니다

식	뜻
`IntersectionObserver`	요소가 화면의 어느 범위에 걸치는지를 알려 주는 브라우저 기능입니다. 스크롤 이벤트를 계속 계산하지 않아 가볍습니다
`rootMargin: '-50% 0px -50% 0px'`	관찰 범위를 위아래 50%씩 잘라 내서 **화면 가운데의 얇은 띠**로 만듭니다. 섹션이 이 띠에 걸치면 "지금 보는 섹션"입니다
`entry.isIntersecting`	그 섹션이 띠에 걸쳐 있는지입니다
`aria-current="true"`	"지금 위치"라는 뜻입니다. 화면 낭독기도 읽고, CSS가 이 속성으로 보라색을 칠합니다
`href.slice(1)`	`#service`에서 `#`를 뗀 `service`입니다

B-3. 왜 이렇게

선택	이유
화면 **가운데 띠** 기준	섹션이 화면 한 장을 꽉 채우므로(`min-height: 100svh`) 가운데에 오는 것이 "보고 있는 것"입니다. 위쪽 기준이면 다음 섹션이 보이기 시작하자마자 바뀝니다
섹션마다 `inside`를 기억하고 표시는 한 번에 계산(`refresh`)	스크롤하면 한 섹션이 나가고 다음이 들어오는 두 변화가 따로 올 수 있습니다. 그때 중간에 두 개가 표시되거나 모두 사라지는 순간이 생기지 않게, 변화를 모두 반영한 뒤 한 번에 정합니다
첫 화면과 섹션 사이에서는 표시 없음	아무 섹션도 아닌 곳에 표시가 남아 있으면 잘못된 안내입니다
링크의 `href`에서 섹션을 찾음	섹션을 추가해도 링크만 더하면 됩니다. 섹션이 없는 링크는 건너뜁니다
`IntersectionObserver`가 없으면 그냥 끝냄	표시만 빠질 뿐 링크 이동은 그대로 됩니다
처음에도 표시가 맞음	관찰을 시작하면 현재 상태가 곧바로 한 번 알려집니다. 예를 들어 `#feature` 주소로 들어와도 맞게 표시됩니다

C. `initMobileMenuClose()` (모바일 메뉴에서 링크를 누르면 닫기)

C-1. 하는 일

폭이 좁은 화면(모바일, 태블릿)에서는 머리글에 `Menu` 버튼이 있고, 누르면 섹션 링크 5개와 로그인 버튼들이 든 상자가 열립니다. 홈에서 링크를 누르면 **그 섹션으로 이동하지만 메뉴가 열린 채로 남아** 화면을 가렸습니다. 이제 메뉴 안의 링크나 버튼을 누르면 메뉴가 닫힙니다.

누른 곳	결과
메뉴 안의 링크 (`Service`, `Character` 등)	메뉴가 닫히고 그 섹션으로 이동
메뉴 안의 버튼 (`Sign In`)	메뉴가 닫히고 로그인 창이 열림
메뉴 안의 링크도 버튼도 아닌 곳	닫히지 않음

C-2. 코드

```
export function initMobileMenuClose() {
  const menus = document.querySelectorAll('details.menu');
  for (const menu of menus) {
    const panel = menu.querySelector('.menu__panel');
    if (!panel) continue;
    panel.onclick = function (event) {
      if (event.target.closest('a, button')) menu.open = false;
    };
  }
}
```

최신 문법으로 쓰면 (공부용)

```
export const initMobileMenuClose = () => {
  document.querySelectorAll('details.menu').forEach((menu) => {
    menu.querySelector('.menu__panel')?.addEventListener('click', ({ target }) => {
      if (target.closest('a, button')) menu.open = false;
    });
  });
};
```

최신 문법	실제 코드	뜻
`?.addEventListener`	`if (!panel) continue;`	요소가 없으면 건너뜁니다
`.forEach((menu) => …)`	`for (const menu of menus)`	목록을 하나씩 처리합니다
`({ target }) =>`	`function (event)` 와 `event.target`	화살표 함수와 구조 분해입니다

식	뜻
`<details class="menu">`	펼치고 접는 HTML 요소입니다. `open` 속성이 있으면 펼쳐진 상태입니다
`menu.open = false`	메뉴를 닫습니다
`event.target.closest('a, button')`	누른 곳에서 위로 올라가며 가장 가까운 링크나 버튼을 찾습니다. 링크 안의 글자를 눌러도 링크를 찾습니다
`panel.onclick = …`	처리기를 하나만 둡니다. 두 번 불러도 겹치지 않고 덮어씁니다

C-3. 왜 이렇게

선택	이유
링크뿐 아니라 **버튼**(Sign In)도 닫음	로그인 창이 메뉴 위에 겹쳐 뜨면 창을 닫은 뒤 메뉴가 그대로 남아 어색합니다
빈 곳을 누르면 닫지 않음	메뉴 안의 여백을 잘못 눌렀다고 닫히면 불편합니다
메뉴 상자(`.menu__panel`)에 하나만 연결	링크와 버튼이 몇 개든 한 곳에서 받습니다
`includePartials()` 뒤에 호출	메뉴는 공통 머리글 조각이라 서버에서 받아 끼운 뒤에야 존재합니다. 이 파일 맨 아래의 호출 순서가 이미 그렇습니다
홈에서만 연결	다른 페이지의 메뉴 링크는 `home.html#…`로 **페이지를 이동**해서 메뉴가 저절로 사라집니다. 같은 문제가 나는 곳은 홈뿐입니다 (다른 페이지의 Sign In 버튼은 로그인 모달이 홈에만 있어 해당 없음)

시험 (실제 서버, 헤드리스 Chrome, 마우스와 키보드로 조작, 29개)

`initIntro` (9개)

#	한 일	결과
I01	새 탭에서 처음 홈을 엶	로딩이 보임 ✓
I02	1.8초 뒤	탭에 `snx-intro = seen`이 기록됨 ✓
I03	로딩의 끝	1초 안팎에 사라짐 ✓
I04	같은 탭에서 홈을 다시 엶	로딩이 보이지 않음 ✓
I05	`DOMContentLoaded` 시점(`home.js`가 헤더를 끼우기 전)	이미 `data-intro="seen"`이고 로딩은 `display: none`, 즉 깜빡임 없음 ✓
I06	두 번째 방문	`data-intro="seen"` 유지 ✓
I07	기록을 지우고 다시 엶 (새 탭과 같음)	로딩이 다시 보임 ✓
I08	저장소가 막힌 환경	오류 없이 열리고 공지 카드까지 채워짐 ✓
I09	저장소가 막힌 채로 다시 엶	매번 로딩이 보임 ✓

`initSideNavActiveState` (13개)

#	한 일	결과
N01	맨 위(첫 화면)	표시 없음 ✓
N02 ~ N06	Service, Character, Feature, Development, Contact 섹션을 차례로 화면 가운데로	각각 그 링크 **하나만** `aria-current` ✓
N07	맨 아래	Contact 유지 ✓
N08	다시 맨 위	표시가 모두 사라짐 ✓
N09	두 섹션의 경계	표시는 하나 이하 ✓
N10	세로 앵커의 `F`를 마우스로 누름	Feature로 이동(부드러운 스크롤 뒤)하고 `F`가 표시됨 ✓
N11	함수를 두 번 더 부름	표시는 하나 이하, 오류 없음 ✓
N12	가리키는 섹션이 없는 링크만 있음	오류 없이 끝남 ✓
N13	`IntersectionObserver`가 없음	오류 없이 끝남 ✓

`initMobileMenuClose` (7개, 폭 390)

#	한 일	결과
B01	`Menu`를 마우스로 누름	메뉴가 열림 ✓
B02	메뉴의 `Character` 링크를 누름	메뉴가 닫히고 `#character`로 이동 ✓
B03	메뉴 상자의 링크도 버튼도 아닌 곳을 누름	닫히지 않음 ✓
B04	메뉴의 `Sign In` 버튼을 누름	메뉴가 닫히고 로그인 창이 열림 ✓
B05	다시 열어서 `Contact` 링크를 누름	닫히고 `#contact`로 이동 ✓
B06	함수를 두 번 더 부르고 다시 시험	정상 동작 ✓
B07	메뉴 요소가 없음	오류 없이 끝남 ✓

29개 모두 통과했습니다. 시험을 만드는 중에 시험 쪽 문제가 몇 번 있었습니다. B03, B04는 직전 단계에서 페이지가 스크롤된 채라 `Menu` 버튼이 헤더에 가려 눌리지 않았던 것이라, 단계마다 홈을 새로 열도록 고쳤습니다. 한 번은 로딩 시험(I01)이 서버가 막 켜진 직후 250ms 안에 로딩을 못 본 적이 있었고, 이후 같은 시험을 세 번 더 돌려 모두 통과했습니다 (시간 여유가 빠듯한 시험입니다). 앱 코드를 고친 곳은 없습니다.

JS 101 — `initTocActiveState()` (개인정보 처리방침 목차에서 지금 읽는 절 표시)와 오래된 TODO 주석 정리

항목	내용
파일	`studionx/frontend/static/js/pages/privacy.js`, `static/css/pages/privacy.css`, `static/js/core/api.js`, `auth.js`, `auth-ui.js`, `pages/home.js`(주석)
작업	P1-23 화면 마무리의 마지막 함수. 목차의 현재 절 표시(선택 기능)와, 구현이 끝났는데 "TODO"로 남아 있던 주석 정리
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	100 홈의 세 함수 · (P1-23 끝, 개요와 시험 문서는 62, 63번)

1. 하는 일

목차의 현재 절 표시

개인정보 처리방침(`privacy.html`)은 12개 절로 길고, 왼쪽(좁은 화면에서는 위)에 목차가 있습니다. 읽는 절에 해당하는 목차 항목에 `aria-current="true"`를 달고, CSS가 글자색과 왼쪽 선(보라색)으로 표시합니다. 마우스를 올렸을 때의 모양과 같습니다.

스크롤 위치	표시되는 목차
맨 위(제목과 첫 문단)	없음
어떤 절의 제목이 화면 위쪽에 옴	그 절 하나만
두 절의 경계가 위쪽 띠에 걸림	아래쪽(막 들어오는) 절
맨 아래	마지막 절(`contact`)

오래된 TODO 주석 정리

`core/api.js`, `auth.js`, `auth-ui.js`에는 함수를 만들기 전 템플릿 때의 `TODO:` 안내가 남아 있었습니다. 구현은 끝났는데도 "TODO"로 보여서 구현이 안 된 것처럼 읽혔습니다. 8곳(`api.js` 1곳, `auth.js` 6곳, `auth-ui.js` 1곳)을 `구현 메모:`로 바꿨습니다. 내용은 그대로이고 코드는 건드리지 않았습니다. `home.js`의 `initLoginForm({ onSuccess: null /* TODO … */ })`는 기본 동작(`next`가 있으면 그 페이지로, 없으면 모달을 닫고 헤더 전환)을 쓰는 것이 맞아서, 주석을 그 설명으로 바꿨습니다. 맨 위의 "템플릿 — 구현은 직접" 안내도 현재 상태에 맞게 고쳤습니다.

2. 언제 쓰나

호출하는 곳	시점
`privacy.js` 맨 아래 `initTocActiveState();`	개인정보 처리방침이 열릴 때 한 번

3. 코드

```
export function initTocActiveState() {
  const links = document.querySelectorAll('.policy__toc a');
  if (links.length === 0 || typeof IntersectionObserver === 'undefined') return;

  const targets = [];   // { link, section, inside }
  for (const link of links) {
    const href = link.getAttribute('href') || '';
    const hash = href.indexOf('#') >= 0 ? href.slice(href.indexOf('#') + 1) : '';
    const section = hash ? document.getElementById(hash) : null;
    if (section) targets.push({ link: link, section: section, inside: false });
  }
  if (targets.length === 0) return;

  const refresh = function () {
    let current = null;
    for (const target of targets) {
      if (target.inside) current = target;   // 띠에 두 절이 함께 걸치면 아래쪽 절
    }
    const atBottom = window.innerHeight + window.scrollY >= document.documentElement.scrollHeight - 2;
    if (atBottom) current = targets[targets.length - 1];
    for (const target of targets) {
      if (target === current) target.link.setAttribute('aria-current', 'true');
      else target.link.removeAttribute('aria-current');
    }
  };

  const observer = new IntersectionObserver(function (entries) {
    for (const entry of entries) {
      for (const target of targets) {
        if (target.section === entry.target) target.inside = entry.isIntersecting;
      }
    }
    refresh();
  }, { rootMargin: '-10% 0px -80% 0px', threshold: 0 });

  for (const target of targets) observer.observe(target.section);
  window.addEventListener('scroll', function () { window.requestAnimationFrame(refresh); }, { passive: t
rue });
}
```

CSS (`privacy.css`, 마우스를 올렸을 때의 규칙에 한 줄 더함)

```
.policy__toc a:hover, .policy__toc a:focus-visible,
.policy__toc a[aria-current="true"] { color: var(--c-ink); border-left-color: var(--c-violet-ink); }
```

최신 문법으로 쓰면 (공부용)

```
export const initTocActiveState = () => {
  const targets = [...document.querySelectorAll('.policy__toc a')]
    .map((link) => ({ link, section: document.getElementById(link.hash.slice(1)), inside: false }))
    .filter((t) => t.section);
  if (!targets.length || !('IntersectionObserver' in window)) return;
```

```
  const refresh = () => {
    const atBottom = innerHeight + scrollY >= document.documentElement.scrollHeight - 2;
    const current = atBottom ? targets.at(-1) : targets.findLast((t) => t.inside);
    targets.forEach((t) => t.link.toggleAttribute('aria-current', t === current));
  };
  const io = new IntersectionObserver((entries) => {
    entries.forEach(({ target: el, isIntersecting }) => { targets.find((t) => t.section === el).inside =
isIntersecting; });
    refresh();
  }, { rootMargin: '-10% 0px -80% 0px' });
  targets.forEach((t) => io.observe(t.section));
  addEventListener('scroll', () => requestAnimationFrame(refresh), { passive: true });
};
```

최신 문법	실제 코드	뜻
`link.hash`	`href.indexOf('#')`와 `slice`	링크 주소에서 `#` 뒤 부분을 바로 꺼냅니다
`targets.at(-1)`	`targets[targets.length - 1]`	마지막 요소
`targets.findLast(…)`	반복에서 마지막으로 덮어쓰기	조건에 맞는 마지막 요소
`toggleAttribute('aria-current', 조건)`	`setAttribute`와 `removeAttribute`	속성을 켜고 끕니다
`.map().filter()`, `{ link, section }`	`for`와 `if`, `{ link: link, … }`	목록 변환과 줄여 쓴 객체

문법 풀이

식	뜻
`rootMargin: '-10% 0px -80% 0px'`	관찰 범위를 위에서 10%, 아래에서 80% 잘라 냅니다. **위에서 10% ~ 20% 사이의 얇은 띠**만 남아서, 그 띠에 걸친 절이 "지금 읽는 절"입니다
`entry.isIntersecting`	그 절이 띠에 걸쳐 있는지입니다
`window.innerHeight + window.scrollY >= document.documentElement.scrollHeight - 2`	화면 아래끝이 문서 끝에 닿았는지(맨 아래인지)입니다. 2는 소수점 오차를 위한 여유입니다
`window.requestAnimationFrame(refresh)`	다음 화면 갱신 직전에 한 번 실행합니다. 스크롤이 초당 수십 번 일어나도 화면 갱신당 한 번만 계산합니다
`{ passive: true }`	이 처리기가 스크롤을 막지 않는다고 브라우저에 알려 스크롤이 매끄럽게 합니다
`href.slice(href.indexOf('#') + 1)`	목차 링크가 `privacy.html#who`처럼 파일 이름을 포함해서, `#` 뒤의 `who`만 뽑습니다 (세로 앵커의 `#service`와 다른 점입니다)

4. 왜 이렇게

선택	이유
화면 **위쪽 띠**(10% ~ 20%) 기준	처리방침은 글을 위에서 아래로 읽는 문서라서, 절 제목이 화면 위쪽에 닿았을 때가 "이 절을 읽기 시작함"입니다. 세로 앵커(홈)는 섹션이 화면 한 장이라 가운데 기준이었습니다
경계에서는 **아래쪽** 절	띠가 얇아도 두 절이 함께 걸칠 수 있습니다. 새로 들어오는 절을 표시하는 것이 자연스럽습니다 (시험에서 위쪽 절을 고르는 첫 구현이 틀렸음을 발견해 고쳤습니다)
맨 아래에서는 마지막 절	마지막 절들은 문서 끝에 막혀서 위쪽 띠까지 올라오지 못합니다. 그대로 두면 `children`, `breach`, `changes`, `contact`는 영영 표시되지 않습니다

선택	이유
스크롤 처리기도 함께 둠	맨 아래 판단은 "절이 띠에 걸쳤는가"와 무관하게 스크롤 위치로만 알 수 있어서, 관찰기가 알려 주지 않는 순간에도 다시 계산해야 합니다
글머리에서는 표시 없음	제목과 첫 문단은 어느 절도 아닙니다
선택 기능이라 실패해도 조용히 끝냄	처리방침은 표시가 없어도 읽는 데 지장이 없어야 합니다. `IntersectionObserver`가 없으면 그냥 끝냅니다
오래된 `TODO`를 `구현 메모`로	문서 내용은 그대로이고, 구현이 안 된 곳처럼 읽히는 것만 고쳤습니다. 코드는 건드리지 않았습니다

5. 시험 (실제 서버, 헤드리스 Chrome, 마우스로 조작, 12개)

#	한 일	결과
P00	목차 링크	12개 ✓
P01	맨 위	표시 없음 ✓
P02	각 절의 제목을 화면 위쪽 띠에 놓음 (스크롤이 닿는 8개 절)	그 절의 목차 **하나만** 표시 ✓
P03	맨 아래로	마지막 절 `contact` 표시 (띠에 닿지 못해도) ✓
P04	다시 맨 위로	표시가 사라짐 ✓
P05	목차의 `rights`를 마우스로 누름	그 절로 이동하고 표시됨 ✓
P06	표시된 항목과 보통 항목	글자색과 왼쪽 선 색이 다름 (CSS) ✓
P07	표시 개수	한 번에 하나 ✓
P08	폭 390 (`rights` 절)	동작 ✓
P09	함수를 두 번 더 부름	표시는 하나 이하, 오류 없음 ✓
P10	가리키는 절이 없는 목차만 있음	오류 없이 끝남 ✓
P11	`IntersectionObserver`가 없음	오류 없이 끝남 ✓

12개 모두 통과했습니다. 처음 돌렸을 때 2개가 실패했습니다. 하나는 시험이 문서 끝에 가까운 절(`children` 이후)을 위쪽 띠까지 스크롤하려 했는데 문서가 거기까지 못 올라가는 것이었고(시험이 닿는 절만 확인하도록 고침), 다른 하나는 절 경계를 띠에 놓았을 때 **위쪽 절이 표시된 것**이었습니다. 이쪽은 시험이 아니라 함수의 선택이 맞지 않아서, 경계에서는 아래쪽 절을 고르도록 함수를 고쳤습니다.

`TODO` 정리 확인: `static/js` 아래에서 남은 "TODO"는 `core/modal.js`(오래된 브라우저용 대체 코드, 필요 없음), `core/sentry-init.js`(프론트 Sentry, Phase 1 범위 밖), 시퀀서 파일들(Phase 2)뿐입니다.